

Nuclear magnetic resonance spectroscopy with language transformers

R. ANDREEV

Notes on the implementation

Criteria for the implementation (from the feedback sheet)

- Reproducible, well-structured, extendable
- Documentation, incl. “handshake”
- Does it do what it says on the tin?

My principles

1. The report is the primary documentation with links to the code
2. Outputs are self-documenting with structured paths and logs
3. Pre-processed inputs are local in the repo
4. Model checkpoints on Zenodo, upload/download scripts in the repo
5. Modularity, separation of concerns, provisions for extensions
6. Unit tests for integrity and as usage examples
7. Landing page, readme files, comments, and quick-start guide
8. Granular commits / commit messages

Criteria for the implementation (from the feedback sheet)

- Reproducible, well-structured, extendable
- Documentation, incl. “handshake”
- Does it do what it says on the tin?

My principles

1. The report is the primary documentation with links to the code
2. Outputs are self-documenting with structured paths and logs
3. Pre-processed inputs are local in the repo
4. Model checkpoints on Zenodo, upload/download scripts in the repo
5. Modularity, separation of concerns, provisions for extensions
6. Unit tests for integrity and as usage examples
7. Landing page, readme files, comments, and quick-start guide
8. Granular commits / commit messages

1. The report is the primary documentation with links to the code

Next, we trained the DistilGPT2 decoder-only language model from its pre-trained state on $F/Q/C/H \rightarrow \text{SMILES}$ with the usual CausalLM cross-entropy token supervision loss, obtaining top-10 accuracy of ~86% on the multimodal spectroscopic dataset with a random 80/10/10 split after pre-filtering by correct sum formula and ignoring stereochemistry.

For a proof-of-concept of our main model, we then trained DistilGPT2 from its pre-trained state on

$$\text{SMILES} \rightarrow \left[\begin{array}{c} \vdots \\ \text{grid} \\ \circ \\ \text{grid} \\ \rightleftharpoons \end{array} \right] \text{GRIMACE} \left[\begin{array}{c} \text{grid} \\ \text{grid} \end{array} \right] \quad (15)$$

with the multi-task objective (14) on the top-1M PubChem SMILES of character length at most 70 with the 80/10/10 “cheese” split (cf. §2.4) for training and evaluation. Evaluating the model in 20-beam search mode, we obtained top-10 accuracy of ~93% on the full test set (where ~2% of SMILES are over 100 characters long) and top-10 accuracy of ~97% on SMILES up to 70 characters long. While this does not prove that the model is able to internalize the molecular structure (because it can “cheat” by looking at the given input SMILES), we observe convergence on all four tasks, in particular, the KL loss dips below 10% within ~30 training epochs and continues to decrease slowly after that.

In order to better understand the multimodal spectroscopy data we trained a $\text{GNN} \rightarrow \text{C/H}$ model that passes a molecule through a GNN and returns an estimate of ^{13}C -NMR and ^1H -NMR spectra with an important simplification: each C/H atom is represented by exactly one peak in its modality (but identical peaks are integrated). This is not the case for annotated spectra, where one might misinterpret noise as peaks or miss smaller/overlapping/broad

2. Outputs are self-documenting with structured paths and logs

Examples:

- `gpt2smiles/training/nmr_to_smiles_v2/runs/lambda/full_alberts_55/20250620-0111_GPT2MultiHeadModel/main.py.log` 
- `gpt2smiles/evaluation/nmr_to_smiles_v2/e_correctness_3_conf/lambda_full_alberts_55_20250620-0111_GPT2MultiHeadModel_checkpoints_last.ckpt/source=data-cheese-split__split=test__at_most=10000__downsize_to=10000__do_chemberta=True__do_func_grps=True/subsample=full/do_incl_fsum=True__do_incl_hsqc=True__do_incl_c_pk=True/is_isomer__score_vs_coverage.png` 

3. Pre-processed inputs are local in the repo

Example:



```
generate_data_cheese_split.py
```

```
data-cheese-split
```

```
|-- shifts_test_000.jsonl.gz  
|-- shifts_train_000.jsonl.gz  
|-- shifts_train_001.jsonl.gz  
|-- shifts_train_002.jsonl.gz  
|-- shifts_train_003.jsonl.gz  
|-- shifts_train_004.jsonl.gz  
|-- shifts_train_005.jsonl.gz  
|-- shifts_train_006.jsonl.gz  
`-- shifts_valid_000.jsonl.gz
```

4. Model checkpoints on Zenodo, upload/download scripts in the repo

Example:



- `zenodo/upload.py --filter full_alberts_55`
- `zenodo/download.py --filter full_alberts_55`

5. Modularity, separation of concerns, provisions for extensions

Example: The folder that defines the main model architecture



- `tokenizer.py`: GPT-2 fast tokenizer + special tokens
- `model.py`: subclasses `DistilGPT2`, adds aux heads, etc.
- `aux_heads.py`: extensible registry of auxiliary heads
- `optimizer_mixin.py`: extras on setting up optimizers/schedulers
- `load_model.py`: robust `load_model_state` with diagnostics
- `tests/...`

6. Unit tests for integrity and as usage examples (5/50 test files shown)

The screenshot displays the PyCharm IDE interface during a test run. The top editor pane shows the source code for the test function `test_horizontal_diversity_of_hidden_states` in `test_gpt2models.py`. The function iterates over a range of `self.T` and checks for horizontal diversity of hidden states using `test_final_vs_partial_context_similarity` and `test_horizontal_diversity_of_hidden_states`.

The middle pane shows the test results. A summary bar indicates "119 tests passed" and "119 tests total: 23 sec 42 ms". Below this, a detailed list of tests is shown, including:

- `test_lazyprogression`: 3 sec 70 ms
- `testGPT2ContextStability`: 7 sec 70 ms
- `test_final_vs_partial_context_similarity`: 1 sec 87 ms
- `test_horizontal_diversity_of_hidden_states`: 1 sec 87 ms
- `test_LaM_heads`: 309 ms
- `testLaMHeadDefinitions`: 309 ms
- `test_activation_and_bias_are_shared`: 7 ms
- `test_checkpoint_v2`: 7 ms
- `test_func_group`: 7 ms
- `test_huact`: 7 ms
- `test_velocity`: 7 ms
- `test_all_required_fields_present`: 7 ms
- `test_head_type_is_valid`: 64 ms
- `test_out_dir_for_func_group_matches_length`: 64 ms
- `test_out_dir_posfix`: 64 ms
- `test_logger_likeLaM_LM_match`: 62 ms
- `test_load_model`: 1 ms
- `test_loadModelState`: 1 ms
- `test_initialize_at_start`: 6 ms
- `test_load_checkpoint`: 1 ms
- `test_load_with_model_prefix`: 0 ms
- `test_missing_keys`: 0 ms
- `test_safetensors_load`: 0 ms
- `test_unexpected_keys`: 0 ms
- `test_model`: 17 sec 524 ms
- `testGPT2MultiHeadModel`: 8 sec 246 ms
- `testLaM_test_functions_defined`: 1 sec 472 ms
- `testLaM_outputs_requires_grad`: 1 sec 518 ms
- `test_custom_token_shapes_match`: 1 sec 422 ms
- `test_forward_output_shapes_match`: 1 sec 416 ms
- `test_forward_without_attention_mask`: 1 sec 472 ms
- `test_parallel_batch_norm_norm`: 1 sec 472 ms
- `test_parallel_respected_in_output_shape`: 1 sec 424 ms
- `test_vocab_reloaded_in_tokenizer`: 1 sec 436 ms
- `test_unwrapped_of_registered_and_grad`: 1 sec 416 ms
- `testGPT2MultiHeadModelLoaded`: 2 sec 972 ms
- `test_save_and_reload_LaM_test_weights`: 2 sec 972 ms
- `test_tokenizer`: 1 sec 932 ms
- `testCustomTokenizer`: 1 sec 944 ms
- `test_at_special_tokens_in_vocab`: 68 ms
- `test_at_special_tokens_tokenizer_property`: 68 ms
- `testLaMHead_definition`: 72 ms
- `test_batch_encode_plus_packing`: 64 ms

The bottom pane shows the file explorer with the project structure, including `gpt2models`, `tests`, and various utility files.

7. Landing page, readme files, comments, and quick-start guide



8. Granular commits / commit messages





